



KHOA CÔNG NGHỆ THÔNG TIN

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC QUỐC GIA HÀ NỘI

**Xây dựng đường cong Elliptic bậc siêu cao
và ứng dụng Chữ ký số**

KHÓA LUẬN TỐT NGHIỆP
NĂM HỌC 2025–2026

Người thực hiện: Trần Mạnh Duy

Giảng viên hướng dẫn: TS. GVC. Lê Phê Đô

Nội dung trình bày

- 01 | Giới thiệu
- 02 | Cơ sở lý thuyết toán học
- 03 | Cài đặt hệ thống
- 04 | Kết luận

ECC là nền tảng bảo mật hạ tầng số

TLS 1.3, SSH, Bitcoin (secp256k1), Ethereum (BLS12-381), chữ ký số điện tử

Thực trạng:

- ▶ Hầu hết hệ thống **tái sử dụng** đường cong sinh sẵn bởi NIST, SECG
- ▶ Tham số dùng “hạt giống” không giải thích được $\Rightarrow$ nguy cơ backdoor
- ▶ Sự kiện Dual_EC_DRBG: NSA bị cáo buộc cài cửa hậu vào PRNG chuẩn NIST
- ▶ Mức bảo mật phổ biến 128-bit — chưa đáp ứng yêu cầu dài hạn NIST 2030+

01 | Giới thiệu

Các hệ thống liên quan



Đường cong	$ p $	Bảo mật	NTT (ν_2)	Minh bạch
secp256k1	256 bit	128-bit	Không	Có
P-256 (NIST)	256 bit	128-bit	Không	Không
BLS12-381	381 bit	128-bit	$\nu_2(r-1) = 32$	Có

Hạn chế của các hệ thống hiện tại:

- ▶ Mức bảo mật cố định 128-bit — không đủ cho kịch bản dài hạn
- ▶ Tham số không minh bạch (NIST P-256: seed không giải thích)
- ▶ BLS12-381: chỉ $\mathbb{F}_r$ hỗ trợ NTT, $\mathbb{F}_p$ thì không

Vấn đề cần giải quyết

1. Tự chủ mật mã

Sinh đường cong bằng quy trình minh bạch, không seed bí ẩn

2. Trần bảo mật

ECC 256-bit $\rightarrow$ 128-bit

Cần 1024-bit $\rightarrow$ **256-bit**

Thách thức

- ▶ Sinh đường cong 1024-bit tổn kém tính toán
- ▶ Phải đảm bảo kháng các tấn công hiện đại (TNFS)
- ▶ Cần chứng minh tính đúng đắn toán học cho mọi phép toán
- ▶ Không dựa vào thư viện mật mã có sẵn

Toán học & Bảo mật

- ▶ Xây dựng kiến trúc số học bignum U1024 tự viết từ đầu
- ▶ Cài đặt phương pháp CM với $D = -3$, kiểm chứng bằng BLS12-381
- ▶ Kháng tấn công MOV, Anomalous, Invalid Curve
- ▶ Kiến trúc hỗ trợ trường tùy ý lên đến 1024-bit

Kỹ thuật & Ứng dụng

- ▶ Độc lập thư viện mật mã bên ngoài (tự triển khai)
- ▶ Hỗ trợ lược đồ chữ ký:
Schnorr, ECDSA
- ▶ Chứng minh tính đúng đắn toán học của các phép toán
- ▶ An toàn trước các tấn công đã biết trên đường cong elliptic

Cơ sở toán học: Trường số nguyên tố $\mathbb{F}_p$

Định nghĩa

$\mathbb{F}_p = \{0, 1, \dots, p-1\}$ với phép cộng và nhân $(\text{mod } p)$,
mọi phần tử $a \neq 0$ có nghịch đảo $a^{-1} = a^{p-2} \pmod{p}$

Tối ưu phép nhân: Dạng biểu diễn Montgomery

- ▶ Chọn $R = 2^{1024}$, biểu diễn $\hat{a} = a \cdot R \pmod{p}$
- ▶ Phép nhân Montgomery (REDC):
 $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod{p}$
- ▶ Thay phép chia cho p bằng
phép chia cho R (dịch bit)

	Thông thường	Montgomery
Rút gọn	$\div p$	$\div R$ (dịch bit)
Chi phí	$O(n^2) + \text{chia}$	$O(n^2)$

Short Weierstrass trên $\mathbb{F}_p$

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

Phép cộng điểm $P_1 + P_2 = P_3$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2$$

Phép nhân vô hướng:

$[k]P$ bằng thuật toán *Double-and-Add*,
độ phức tạp $O(\log k)$

Bài toán logarit rời rạc (ECDLP)

- ▶ Cho $P, Q = [k]P$, tìm k là bài toán khó (cơ sở bảo mật)
- ▶ Độ phức tạp: $O(\sqrt{r})$ với thuật toán Pollard's rho
- ▶ Nhóm con cấp r nguyên tố đảm bảo an toàn

Phương pháp Nhân phức (Complex Multiplication)

Điều kiện CM: Tồn tại đường cong trên $\mathbb{F}_p$ với vết Frobenius t khi:

$$4p = t^2 + |D| \cdot y^2 \quad (D < 0 \text{ là biệt thức CM})$$

Quy trình 4 bước:

1. Tính j -invariant từ đa thức lớp Hilbert $H_D(x) \pmod{p}$
2. Suy ra hệ số đường cong (a, b) từ j
3. Kiểm tra xoắn (twist test): chọn đường cong có $r \mid \#E$
4. Tìm điểm sinh G bậc r bằng nhân cofactor

Schnorr — 384 byte

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$e = H(R_x \| R_y \| m) \bmod r$

$s = k + e \cdot d \pmod{r}$

Chữ ký: $\sigma = (R, s)$

Xác minh: $[s]G \stackrel{?}{=} R + [e]Q$

ECDSA (FIPS 186-4) — 256 byte

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$r_{\text{sig}} = R_x \bmod n$, $e = H(m)$

$s = k^{-1}(e + r_{\text{sig}} \cdot d) \bmod n$

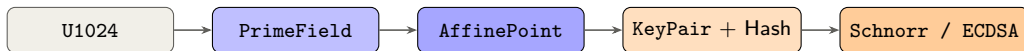
Chữ ký: (r_{sig}, s)

Xác minh: $[u_1]G + [u_2]Q$, kiểm tra P_x

03 | Cài đặt hệ thống



Công cụ dòng lệnh curve1024-sig



Lệnh CLI

```
keygen — sinh cặp khóa  
sign -f <file> -scheme  
schnorr|ecdsa  
verify -f <file> -k <pub>  
export-pub — xuất khóa công khai
```

Đặc điểm

- ▶ Tham số đường cong cấu hình qua file TOML
- ▶ Không phụ thuộc thư viện mật mã bên ngoài
- ▶ Hỗ trợ thay đổi đường cong mà không sửa mã nguồn

03 | Cài đặt hệ thống



Tham số đường cong: BLS12-381 làm baseline

Cấu hình config/curve1024.toml

Trường cơ sở p : 381-bit Bậc nhóm r : 255-bit $a = 0$, $b = 4$, $D = -3$, $k = 12$

Lý do sử dụng BLS12-381 làm baseline:

- ▶ Đường cong chuẩn công nghiệp, đã được kiểm chứng độc lập bởi cộng đồng quốc tế
- ▶ Cho phép so sánh kết quả với thư viện tham chiếu zkcrypto/bls12_381
- ▶ Kiến trúc U1024 hỗ trợ tham số tùy ý — chỉ cần thay file curve1024.toml

Kết quả kiểm chứng: 36/36 test PASSED trên tham số BLS12-381, xác nhận tính đúng đắn của toàn bộ ngăn xếp thư viện.

03 | Cài đặt hệ thống

So sánh hiệu năng



Đường cong	$ p $	Ký	Xác minh	Bảo mật
secp256k1	256	~ 0.2 ms	~ 0.4 ms	128-bit
P-384	384	~ 1 ms	~ 2 ms	192-bit
BLS12-381	381	~ 1.5 ms	~ 3 ms	128-bit
Curve1024	1024	1.55 s	2.83 s	256-bit

Schnorr sign/verify — Criterion, release build, LLVM O3.
Đo trên phần mềm thuần Rust, chưa tối ưu ASM.

Nguyên nhân chậm:

- ▶ Trường $\mathbb{F}_p$ lớn gấp $4\times$ secp256k1
 $\Rightarrow$ nhân chậm $\sim 16\times$ (bậc hai)
- ▶ Chưa có Montgomery Ladder, tọa độ Projective, inline ASM

Kịch bản phù hợp:

- ▶ Ký tài liệu pháp lý, phần mềm
- ▶ Chứng chỉ PKI dài hạn
- ▶ Hệ thống cần bảo mật hậu lượng tử mức 256-bit

Kết quả

- ✓ Kiến trúc bignum U1024 tự viết, an toàn bộ nhớ (Rust)
- ✓ Số học Montgomery, phép toán nhóm Short Weierstrass đúng đắn
- ✓ Phương pháp CM ($D = -3$), kiểm chứng chéo BLS12-381
- ✓ Schnorr & ECDSA hoạt động đúng, 36/36 test PASSED
- ✓ Miễn nhiễm Anomalous ($\#E \neq p$) và Invalid Curve

Hạn chế

- ▶ **Timing side-channel**
Double-and-Add rò rỉ bit scalar
⇒ cần Montgomery Ladder
- ▶ **Tọa độ Affine**
1 nghịch đảo/phép cộng điểm
⇒ cần Jacobian/Projective
- ▶ **Hiệu năng**
~8000× chậm hơn secp256k1
(trường lớn 4× + chưa tối ưu ASM)

Hướng phát triển

1. Tối ưu hiệu năng

Tọa độ Projective	1 nghịch đảo/lần nhân vô hướng (dự kiến $\sim 10\times$)
Montgomery Ladder	Constant-time — loại bỏ timing side-channel
Sliding Window	Giảm $\sim 25\%$ số phép cộng điểm

⇒ Thu hẹp khoảng cách hiệu năng với secp256k1 từ $8000\times$ xuống $\sim 500\times$

2. Chữ ký BLS & xác minh gộp

Cài đặt phép ghép cặp (Miller loop + final exponentiation trên $\mathbb{F}_{p^{18}}$)

⇒ Khai thác embedding degree $k=18$ — ưu thế cốt lõi của KSS18

3. Kiểm định hình thức

Chứng minh tính đúng đắn bằng Creusot/Kani cho Rust

⇒ Đảm bảo mức tin cậy cần thiết cho triển khai mật mã thực tế

Cảm ơn thầy cô đã lắng nghe!